Tworzenie aplikacji z użyciem React

Workbook uczestnika

Język: PL

Wybrane opcje: redux, zustand, mobx

Agenda

1. Wprowadzenie
2. Podstawy React
3. React Hooks
4. React w praktyce
5. Zaawansowany React
6. Praca z wybranym mechanizmem zarządzania stanem aplikacji
7. Testowanie (opcja)

Przygotowanie do pracy

- repozytorium ćwiczeniowe: github.com/matwjablonski/react-open-10-26
- edytor kodu z terminalem
- aktualna wersja Node.js i npm
- przeglądarka z narzędziami developerskimi

Zadania tworzą jedną rozwijaną aplikację. Zachowuj działający stan projektu po każdym ćwiczeniu i zapisuj decyzje, które chcesz omówić.

Zadanie nr 1

- sklonuj repozytorium szkoleniowe i uruchom przygotowany projekt React + TypeScript

```
git clone https://github.com/matwjablonski/react-open-10-26.git
cd react-open-10-26/my-books-app
npm ci
npm run dev
```

- zostaw aplikację uruchamianą przez `src/main.tsx` i główny komponent `src/App.tsx`
- zmodyfikuj `App.tsx`, tak aby wyświetlał informacje o aplikacji:

1. tytuł: "Witaj w naszej aplikacji książkowej"
2. polecane książki: listę `ul / li` a w niej 3 tytuły książek
3. stopkę z informacją: "Aplikacja przygotowana przez <imię i nazwisko>"

- przygotuj plik `src/data/books.ts` z tablicą minimum 3 książek, które wykorzystamy w kolejnych zadaniach
- każda książka powinna zawierać stabilne `id`, `title`, `author` oraz `publicationDate`

Notatki i wnioski

Zadanie nr 2

- wydziel tytuł strony do osobnego komponentu `Header`
- wydziel stopkę strony do komponentu `Footer`
- przygotuj komponenty `Books` oraz `Book`
- komponent `Books` powinien renderować książki z pliku `src/data/books.ts` przekazane przez `props`
- komponent `Book` powinien wyświetlić tytuł, autora oraz datę publikacji - dane te powinien otrzymać z `props`
- każda książka powinna mieć stabilne `id`, które wykorzystamy później w routingu i stanie aplikacji

Notatki i wnioski

Zadanie nr 3

- w komponencie `App` dodaj stan przechowujący identyfikatory przeczytanych książek
- do komponentu `Book` dodaj przycisk, który będzie miał dwa stany: "Dodaj do przeczytanych" lub "Usuń z przeczytanych"
- po kliknięciu w przycisk stan powinien się zaktualizować, a obok tytułu książki powinna pojawić się informacja, że książka została przeczytana
- do wykonania zadania wykorzystaj `useState`, `props` i `onClick`

Notatki i wnioski

Pytania po zadaniu

> Który komponent powinien być właścicielem tego stanu i dlaczego?

> Która tożsamość książki musi pozostać stabilna po zmianie kolejności lub usunięciu?

> Jak sprawdzić, że aktualizacja jednej książki nie zmienia pozostałych?

Notatki i wnioski

Zadanie nr 4

- za pomocą TypeScript opisz propsy wchodzące do komponentów `Book` oraz `Books`
- przygotuj typ `BookItem`, który opisze dane jednej książki
- komponent `Books` powinien przyjmować tablicę `BookItem[]`
- komponent `Book` powinien przyjmować dane jednej książki, informację `isRead` oraz callback do zmiany stanu przeczytania
- wyeksportuj typy z jednego miejsca, np. `src/types/book.ts`

Notatki i wnioski

Zadanie nr 5

- przygotuj komponenty `Input` oraz `Form`
- komponent `Input` powinien przyjmować wszystkie standardowe atrybuty elementu `input` oraz prop `label`
- komponent `Form` powinien przyjmować `onSubmit` oraz `children`
- na podstawie tych komponentów przygotuj `ContactForm`, który umieścisz w stopce
- wysłanie formularza ma na razie wypisać dane w konsoli przeglądarki
- formularz powinien zawierać pola `name`, `message`, `phoneNumber`

Notatki i wnioski

Zadanie nr 6

- załóż, że nagłówek `Header` może przyjąć prop `children`
- przygotuj komponent `UserInfo`, który wyświetli informację o zalogowanym użytkowniku
- przekaż `UserInfo` do `Header` przez `children`

Notatki i wnioski

Zadanie nr 7

- sprawdź wysokość listy książek (użyj `useRef` oraz `getBoundingClientRect`)
- jeśli wysokość jest wyższa niż 500px to wypisz nad listą informację: "Masz masę książek"
- jeśli jest mniejsza to wypisz "Zbieraj dalej"
- wykonaj pomiar po wyrenderowaniu listy, a nie w trakcie renderowania komponentu

Notatki i wnioski

Zadanie nr 8

- do napisanego wcześniej formularza kontaktowego dodaj przycisk "Napisz do nas", który po kliknięciu "postawi" kursor w pierwszym polu formularza
- użyj do wykonania tego zadania `useRef` i `.focus()`
- użyj `useRef` w komponencie rodzica i przekaż `ref` do komponentu pola formularza jako zwykły prop `ref`

Notatki i wnioski

Zadanie nr 9

- do każdej książki dodaj przycisk "Głosuj na tę pozycję"
- na liście pokaż informację, ile głosów oddano na daną pozycję
- użyj `useState` z inicjalizatorem funkcyjnym
- wartość początkowa powinna wynikać z operacji `<liczba liter w nazwisku autora> * 2 / 5`
- pamiętaj, że liczba głosów musi być liczbą całkowitą, użyj `Math.round()` albo `Math.floor()`

Notatki i wnioski

Zadanie nr 10

- zmień tytuł okna przeglądarki (`document.title`) tak, aby zawierał liczbę książek dostępnych w naszej aplikacji
- tytuł powinien brzmieć: "Twój zbiór książek liczy książek"
- wykorzystaj `useEffect`
- zadbaj o poprawną tablicę zależności i sprawdź zachowanie w `StrictMode`
- po odmontowaniu komponentu przywróć domyślny tytuł strony

Notatki i wnioski

Pytania po zadaniu

> Z jakim zewnętrznym systemem synchronizuje się ten efekt?

> Które zachowanie w StrictMode ujawni brak idempotencji lub poprawnego cleanupu?

> Kto powinien przywrócić poprzedni stan, gdy kilka widoków korzysta z tego samego zasobu?

Notatki i wnioski

Zadanie nr 11

- dodaj funkcjonalność usuwania książek
- obok przycisku przeczytane dodaj przycisk "Usuń"
- po kliknięciu książka powinna zniknąć z listy a lista się odświeżyć
- przebuduj zarządzanie listą książek z `useState` na `useReducer`
- przygotuj akcje dla oznaczania książki jako przeczytanej, cofania oznaczenia i usuwania książki
- zamodeluj akcje jako union type w TypeScript

Notatki i wnioski

Zadanie nr 12

- endpoint `https://jsonplaceholder.typicode.com/users` zwraca listę użytkowników (czytelnicy)
- przygotuj komponent `Readers`, który pobierze i wyświetli listę czytelników naszej aplikacji
- lista powinna mieć 2 stany: ukryty (wyświetla tylko informację o liczbie użytkowników) oraz rozwinięty (wyświetla listę użytkowników)
- pamiętaj o dodaniu przycisku, który będzie ukrywał i pokazywał listę
- na liście rozwiniętej powinny być widoczne takie dane jak imię, email, username, website
- użyj `useEffect` i `fetch` do pobrania danych
- obsłuż stan ładowania, błąd oraz anulowanie zapytania przez `AbortController`

Notatki i wnioski

Pytania po zadaniu

> Co powinno się wydarzyć, gdy starsze zapytanie zakończy się po nowszym?

> Jak odróżnić anulowanie operacji od błędu widocznego dla użytkownika?

> Które kombinacje stanów ładowania, danych i błędu powinny być niemożliwe?

Notatki i wnioski

Zadanie nr 13

- napisz swój własny hook do pobierania danych z API
- hook powinien zwracać `data`, `isLoading` oraz `error`
- hook powinien przyjmować endpoint jako argument
- wykorzystaj go w widoku czytelników z poprzedniego zadania
- zadbaj o typ generyczny, np. `useApi<Reader[]>('/users')`
- na tym etapie bazowy adres API może być stałą wewnątrz hooka; przeniesiemy go do konfiguracji środowiska w zadaniu 16

Notatki i wnioski

Zadanie dodatkowe – TanStack Query

- zainstaluj TanStack Query w projekcie `npm install @tanstack/react-query`
- dodaj `QueryClientProvider` do drzewa komponentów, aby udostępnić klienta zapytań w całej aplikacji
- przepisz pobieranie czytelników z API, aby korzystało z hooka `useQuery` z TanStack Query
- dodaj obsługę stanu ładowania i błędów, wykorzystując `isPending`, `isError` oraz `error` zwrócone z `useQuery`
- sprawdź w React Query Devtools lub w zakładce Network różnicę pomiędzy odczytem danych z cache a odświeżeniem w tle
- zostaw wcześniejszy własny hook jako punkt porównania, ale nie używaj go już w widoku czytelników

Notatki i wnioski

Zadanie nr 14

- zainstaluj React Router `npm i react-router`
- dodaj `BrowserRouter` w `src/main.tsx`
- przygotuj ścieżki dla: strony głównej (informacji o tym, co to za aplikacja), listy książek, pojedynczej książki i formularza kontaktowego
- strona główna powinna mieć ścieżkę `/`
- lista książek `/books`
- pojedyncza książka `/books/:id`
- formularz kontaktowy `/contact`
- przenieś listę książek z `App` do widoku `BooksPage`
- przygotuj wspólny `Layout` z nagłówkiem, stopką i miejscem na zagnieżdżone widoki
- użyj nested routes oraz komponentu `<Outlet />`
- dodaj nawigację w nagłówku aplikacji wykorzystując komponent `<Link>` z `react-router`

```
<Link to="/">Home</Link>
```

Notatki i wnioski

Zadanie nr 15

- dodaj do projektu Material UI: https://mui.com/material-ui/getting-started/

```
npm install @mui/material @emotion/react @emotion/styled
```

- jeśli używasz Styled Components, zainstaluj też `styled-components`

```
npm install styled-components
```

- z wykorzystaniem MUI dodaj komponent `Avatar`, aby wyświetlić zdjęcie użytkownika `https://placehold.co/150`
- za pomocą `CSS modules` ostyluj listę książek
- za pomocą `Styled Components` ostyluj nawigację, stopkę oraz zmień styl awatara tak, aby otrzymał dodatkową ramkę wkoło
- zostaw style lokalne przy komponentach, których dotyczą

Notatki i wnioski

Zadanie nr 16

- dodaj plik `.env.local`
- zadeklaruj zmienną środowiskową `VITE_API_URI` z wartością `https://jsonplaceholder.typicode.com`
- przebuduj własny hook do wykonywania zapytań tak, aby pobierał bazowy adres z `import.meta.env.VITE_API_URI`
- do hooka przekazuj ścieżkę, np. `/users`, oraz opcjonalne źródło: `api` albo `public`; domyślnym źródłem powinno być `api`
- wydziel budowanie pełnego adresu do funkcji `src/utils/buildResourceUrl.ts`, którą wykorzystasz później w testach
- dla źródła `api` funkcja powinna dodać bazowy adres z `VITE_API_URI`, a dla `public` pozostawić lokalną ścieżkę bez zmian
- sprawdź oba warianty, pobierając czytelników przez `useApi<Reader[]>('/users')` oraz przygotowany plik przez `useApi<BookItem[]>('/books.json', 'public')`

Notatki i wnioski

Zadanie nr 17

- przygotuj zabezpieczoną ścieżkę dla widoku z czytelnikami (jeśli nie masz takiego widoku dodaj nowy `route`)
- dodaj tymczasowy stan logowania w aplikacji, np. `isAuthenticated`
- jeśli nastąpi próba wejścia na widok czytelników bez uprzedniego zalogowania, przekieruj użytkownika na widok z informacją, że nie ma uprawnień do przeglądania tej strony
- po zalogowaniu użytkownik powinien móc wejść na zabezpieczony widok
- jeśli korzystasz z `createBrowserRouter`, dodaj `errorElement` dla błędów routingu
- potraktuj to wyłącznie jako ochronę interfejsu: backend nadal musi niezależnie sprawdzać uwierzytelnienie i uprawnienia dla każdego żądania

Notatki i wnioski

Pytania po zadaniu

> Przed którymi działaniami nie chroni zabezpieczenie trasy w interfejsie?

> Co backend musi zweryfikować niezależnie dla każdego żądania?

> Jak zachować zamiar użytkownika po zalogowaniu, nie ufając ślepo adresowi przekierowania?

Notatki i wnioski

Zadanie nr 18

- w pliku `index.html` dodaj element, do którego wyrenderujesz modal, np. `<div id="modal-root"></div>`
- z użyciem React Portal przygotuj modal, który będzie wyświetlał się po kliknięciu "Zaloguj"
- logowanie odbędzie się dopiero po kliknięciu "Zaloguj" w modalu
- oznacz modal semantycznie przez `role="dialog"`, `aria-modal="true"` i dostępną nazwę
- po otwarciu przenieś focus do modala i nie pozwól, aby nawigacja klawiaturą wychodziła poza jego interaktywne elementy
- umożliw zamknięcie modala klawiszem Escape
- po zamknięciu modala focus powinien wrócić do przycisku otwierającego

Notatki i wnioski

Pytania po zadaniu

> Co portal zmienia w DOM, a czego nie zmienia w drzewie Reacta?

> Gdzie powinien trafić focus, jeśli element otwierający przestał istnieć?

> Jak sprawdzić obsługę modala bez polegania wyłącznie na myszy?

Notatki i wnioski

Zadanie nr 19

- w naszej aplikacji chcemy obsłużyć dwa motywy: `dark` i `light`
- przygotuj `ThemeContext`, który udostępni aktualny motyw i funkcję jego zmiany
- dodaj przycisk w nagłówku do zmiany motywu
- w stopce dodaj informację o aktualnie wybranym motywie
- zadbaj o to, żeby hook używany poza providerem zwracał czytelny błąd

Notatki i wnioski

Zadanie nr 20

- wykorzystując code splitting podziel aplikację według widoków routingu
- użyj `React.lazy` oraz `<Suspense />`
- przygotuj fallback widoczny podczas ładowania widoku
- lazy loading zastosuj na poziomie stron, np. `BooksPage`, `BookDetailsPage`, `ContactPage`
- upewnij się, że błąd ładowania widoku nie psuje całej aplikacji - użyj route error boundary albo osobnego fallbacku błędu

Notatki i wnioski

Zadanie nr 21

- użyj React DevTools Profiler albo React Performance Tracks w Chrome DevTools, aby zidentyfikować kosztowny render lub obliczenie
- jeśli filtrowanie książek jest rzeczywiście kosztowne, użyj `useMemo` i ponownie wykonaj pomiar
- jeśli niestabilny callback powoduje ponowne renderowanie zmemoizowanego komponentu, ustabilizuj go przez `useCallback` i porównaj wynik
- jeżeli pomiar nie pokazuje problemu, pozostaw kod bez dodatkowej memoizacji i zapisz ten wniosek

Notatki i wnioski

Pytania po zadaniu

> Który pomiar stanowi wiarygodny punkt odniesienia przed optymalizacją?

> Jakie koszty lub ryzyko nieaktualnych danych wprowadza memoizacja?

> Kiedy wynik pomiaru uzasadnia pozostawienie prostszego kodu?

Notatki i wnioski

Zadanie nr 22

- zainstaluj Redux Toolkit i React Redux

```
npm install @reduxjs/toolkit react-redux
```

- skonfiguruj Redux Toolkit w naszej aplikacji
- dodaj `Provider` z `react-redux` w `src/main.tsx`
- przygotuj slice do zarządzania zalogowanym użytkownikiem
- przygotuj typowane hooki `useAppSelector` i `useAppDispatch`
- użyj danych użytkownika w nagłówku oraz w stopce
- do wysłania akcji logowania i wylogowania użyj `useAppDispatch`
- nie przenoś do Reduxa danych pobieranych przez TanStack Query

Notatki i wnioski

Zadanie nr 23

- dodaj selektor zwracający informację, czy użytkownik jest zalogowany
- dodaj selektor zwracający nazwę użytkownika albo tekst `Gość`
- użyj selektorów w nagłówku i stopce zamiast odczytywać cały fragment stanu
- sprawdź w Redux DevTools, jakie akcje zostały wysłane podczas logowania i wylogowania
- sprawdź, które komponenty renderują się po zmianie stanu użytkownika

Notatki i wnioski

Zadanie dodatkowe – Zustand

- zainstaluj `zustand` w projekcie
  `npm install zustand`
- stwórz globalny store dla client state aplikacji, który:
  - przechowuje listę przeczytanych książek (np. jako tablicę ID)
  - pozwala oznaczyć książkę jako przeczytaną
  - pozwala cofnąć oznaczenie
  - pozwala sprawdzić, czy dana książka została przeczytana
- użyj selektorów w komponentach, zamiast pobierać cały store

Notatki i wnioski

Zadanie dodatkowe – Zustand middleware

- dodaj middleware `devtools`, aby móc korzystać z rozszerzenia Redux DevTools do debugowania stanu store'a
- dodaj middleware `persist` do stworzonego wcześniej store'a, aby przechowywać dane w `localStorage` - pamiętaj o odpowiedniej konfiguracji, aby zapisać tylko potrzebne dane (np. listę przeczytanych książek, a nie całą logikę store'a)

Notatki i wnioski

MobX — notatki z modułu

Ten moduł nie ma odrębnego zadania praktycznego. Zapisz obserwacje, które pomogą Ci porównać MobX z pozostałymi mechanizmami zarządzania stanem.

Notatki i wnioski

Zadanie nr 24

- zainstaluj Vitest, Testing Library, `@testing-library/jest-dom`, `@testing-library/user-event` oraz `jsdom`

```
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

- skonfiguruj Vitest w `vite.config.ts`
- dodaj skrypty `test` oraz `test:run` w `package.json`
- przetestuj jedną funkcję z `utils/`
- przetestuj jeden komponent przez zachowanie widoczne dla użytkownika
- użyj `userEvent` do interakcji zamiast ręcznie wywoływać eventy
- jeśli komponent pobiera dane, zainstaluj MSW i mockuj odpowiedź HTTP przez handler

```
npm install -D msw
```

Notatki i wnioski

Pytania po zadaniu

> Które asercje opisują zachowanie widoczne dla użytkownika?

> Gdzie powinna przebiegać granica mockowania komunikacji HTTP?

> Którego istotnego ryzyka nadal nie obejmują przechodzące testy?

Notatki i wnioski